COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	09/16/26	**DATE SUBMITTED:**	09/16/26
NAME:	Prince Benjamin S. Villaruel	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 9.1 Implementing Trees

1. PROCEDURE OUTPUT

What procedures did you perform?

Code Snippet

main.cpp

```cpp
#include <iostream>
#include "binaryTree.h"

main(){


    //creating the root node:
    BinaryTree<char>* root = new BinaryTree<char>('P');

    //creating a child of root node P:
    root->insertLeftChild('R');
    root->insertRightChild('I');

    //insert a child to node R
    root->getLeftChild()->insertLeftChild('N');
    root->getLeftChild()->insertRightChild('C');

    //insert a child to node I
    root->getRightChild()->insertLeftChild('E');

    //print the root node:
    std::cout << "Root: " << root->getRootVal() << std::endl;

    //children of the root node
    std::cout << "root left child: " << root->getLeftChild()->getRootVal() << std::endl;
    std::cout << "root right child: " << root->getRightChild()->getRootVal() << std::endl;

    // children of the sub trees R and I

    std::cout << "node R left child: " << root->getLeftChild()->getLeftChild()->getRootVal() << std::endl;
    std::cout << "node R right child: " << root->getLeftChild()->getRightChild()->getRootVal() << std::endl;

    root->getLeftChild()->getRightChild()->setRootVal('E');
    std::cout << "node R right child: " << root->getLeftChild()->getRightChild()->getRootVal() << std::endl;

    return 0;

}
```

main.cpp ● binaryTree.h ×

binaryTree.h > ...

```h
#ifndef BINARYTREE_H
#define BINARYTREE_H

template <typename T>
class BinaryTree{
    private:
        T key;
        //pointers
        BinaryTree<T>* leftChild;
        BinaryTree<T>* rightChild;

    public:

        //constructor
        BinaryTree(T rootObj);

        //inserting a child node:
        void insertLeftChild(T rootObj);
        void insertRightChild(T rootObj);

        BinaryTree<T>* getLeftChild();
        BinaryTree<T>* getRightChild();

        T getRootVal();
        void setRootVal(T obj);

        bool isLeaf();
    };

template <typename T>
BinaryTree<T>::BinaryTree (T rootObj) {
    key = rootObj;
    leftChild = nullptr;
    rightChild = nullptr;

}

template <typename T>
void BinaryTree<T>::insertLeftChild(T newNode) {

    if(leftChild == nullptr){
        leftChild = new BinaryTree<T>(newNode);
    }
    else{

    //create a new node
    BinaryTree<T> *newChild = new BinaryTree<T> (newNode);

    //move the existing left child below the new Node
    newChild->leftChild = leftChild;
    leftChild = newChild;
    }

}
```

```
56
57  template <typename T>
58  void BinaryTree<T>::insertRightChild(T newNode) {
59      if(rightChild == nullptr){
60          rightChild = new BinaryTree<T>(newNode);
61      }
62      else{
63
64      //create
65      BinaryTree<T> *newChild = new BinaryTree<T> (newNode);
66
67      //move the existing right child below the new Node
68      newChild->rightChild = rightChild;
69      rightChild = newChild;
70      }
71  }
72
73  template <typename T>
74  BinaryTree<T>* BinaryTree<T>::getLeftChild(){
75      return leftChild;
76  }
77
78  template <typename T>
79  BinaryTree<T>* BinaryTree<T>::getRightChild(){
80      return rightChild;
81  }
82
83  template <typename T>
84  T BinaryTree<T>::getRootVal(){
85      return key;
86  }
87
88  template <typename T>
89  void BinaryTree<T>::setRootVal(T obj) {
90      key = obj;
91  }
92
93  template <typename T>
94  bool BinaryTree<T>::isLeaf() {
95      return leftChild == nullptr && rightChild == nullptr;
96  }
97
98
99
100 #endif
```

Output

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\SW 9.1>
Root: P
root left child: R
root right child: I
node R left child: N
node R right child: C
node R right child: E
```

Documentation

For our SW #9.1, we implemented the BinaryTree class to construct and manage a binary tree structure. The constructor sets up a node's data while keeping its child pointers null. We also implemented insertLeftChild and insertRightChild to dynamically attach new nodes or insert them into existing subtrees, complemented by getLeftChild and getRightChild to browse through the tree. To interact with node data, getRootVal retrieves the current value,

	and setRootVal allows updating it on the fly. Lastly, isLeaf provides a quick check to determine whether a node has any children.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.